

PostgreSQL Database Design

ServiceGuard AI

1. Entity relationship

```
customers
■
■■■ applications
■
■■■ services
■
■■■ service_instances
■
■■■ telemetry
■
■■■ incidents
■■■ risk_assessments
■■■ recommended_actions

runbook_documents
■
■■■ runbook_chunks (pgvector)

agent_sessions → agent_tool_calls
audit_logs
```

2. Core constraints

- UUID primary keys for major domain entities.
- Unique customer email and service identifiers where applicable.
- Foreign keys enforce ownership relationships.
- CHECK constraints for percentages/ranges.
- Unique incident idempotency key to prevent duplicate incidents.

3. Important indexes

```
telemetry(service_id, timestamp DESC)
telemetry(instance_id, timestamp DESC)
incidents(status, severity, detected_at DESC)
services(customer_id, status)
risk_assessments(incident_id, created_at DESC)
agent_tool_calls(session_id, timestamp)
audit_logs(entity_type, entity_id, timestamp DESC)
```

4. Semantic views

```
CREATE VIEW ai_service_health AS
SELECT
service_id,
timestamp,
response_time_ms,
error_rate,
cpu_usage_pct,
memory_usage_pct,
db_connections_pct,
availability_pct,
sla_target_ms
FROM telemetry;
```

5. Vector storage

Use pgvector for runbook embeddings in the initial version. Keep document metadata relational so filters such as version, service category and effective date can be enforced before/alongside semantic retrieval.